

Battery Monitor

Aim

Components

Connections - Circuit Diagram

Detailed Steps - Launching Code

Task - Exercise

Aim

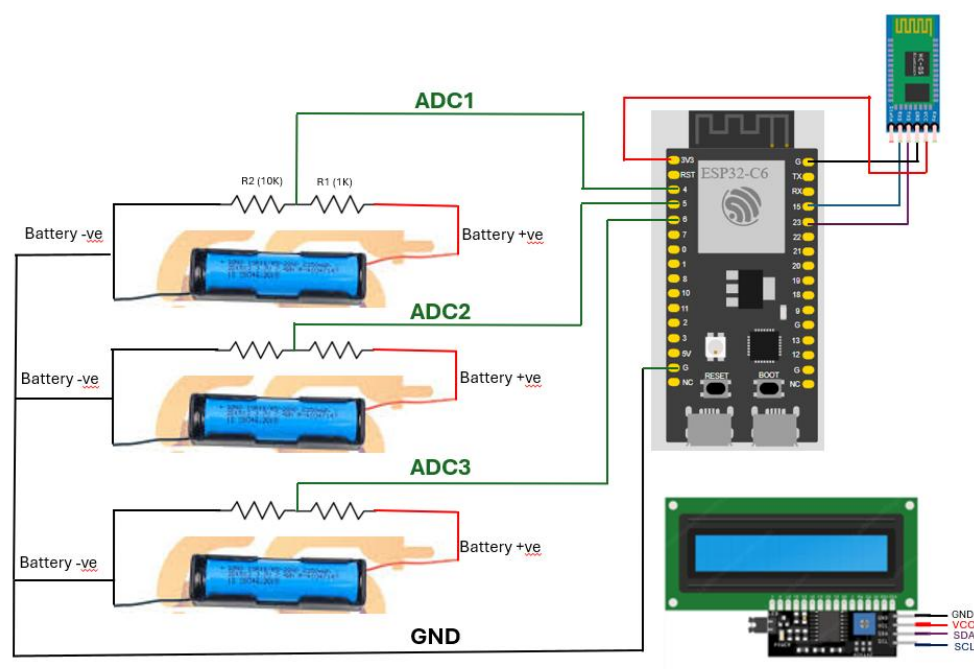
The battery pack in an electric vehicle needs to be continuously monitored to ensure safe, efficient, and reliable operation. This includes measuring key parameters such as battery voltage and converting these analog signals into digital data for processing using a Microcontroller. This build exercise aims to learn the design of a simple Battery Monitoring System for EV applications using an Analog-to-Digital Converter (ADC), implemented through a physical setup in an open-loop configuration.

Components

1. ESP32C6 Dev Module – 1
2. Battery Pack (3s1p) – 1
3. Battery holder - 3
4. 1K Resistor – 3
5. 10K Resistor – 3
6. Jumper wires – As needed

Note: All the components are reusable.

Circuit Diagram



Detailed Steps

Step 1: Place the Parts

- Put the ESP32-C6 board on the breadboard.
- Keep three batteries separate:
 - Battery 1
 - Battery 2
 - Battery 3
- For each battery, place two resistors near it:
 - $R1 = 1\text{ k}\Omega$
 - $R2 = 10\text{ k}\Omega$
- Place the Bluetooth module near the ESP32.
- Place the 16×2 I2C LCD module.

Step 2: Connect All Grounds

- Connect negative (–VE) terminals of all three batteries together.
- Connect this common line to:
 - ESP32 GND
 - One end of all 10 k Ω resistors ($R2$)
 - Bluetooth GND
 - LCD GND
- All grounds must be connected.

Step 3: Connect Battery Positive

- For each battery:
- Connect battery +VE to one end of 1 k Ω resistor ($R1$).
- Do not connect battery +VE directly to ESP32.

Step 4: Make Voltage Divider

- Connect $R1$ (1 k Ω) and $R2$ (10 k Ω) in series.
- The point where $R1$ and $R2$ meet is used to measure voltage.

- This reduces battery voltage to a safe level for ESP32.

Step 5: Connect ADC Pins

- Battery 1 midpoint to ESP32 GPIO4 (ADC1)
- Battery 2 midpoint to ESP32 GPIO5 (ADC2)
- Battery 3 midpoint to ESP32 GPIO6 (ADC3)

Step 6: Connect Bluetooth Module

- Bluetooth TX to ESP32 GPIO23
- Bluetooth RX to ESP32 GPIO15
- Bluetooth VCC to ESP32 3.3V
- Bluetooth GND to GND

Step 7: Connect I2C LCD

- LCD VCC to ESP32 3.3V
- LCD GND to ESP32 GND
- LCD SDA to ESP32 GPIO21
- LCD SCL to ESP32 GPIO22

Step 8: Power the ESP32

- Power ESP32-C6 using a USB cable.
- Do not power ESP32 directly from batteries.

Step 9: Check Voltage Safety

- Make sure ADC voltage is below 3.3V.
- Battery voltage is reduced by the resistors.
- Never connect battery directly to ADC pin.

Step 10: Final Checks

- Battery + and – are correct
- No short circuit
- Resistors are correct (1 k Ω and 10 k Ω)
- All grounds are connected together

Full Code:

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>

// LCD setup
LiquidCrystal_I2C lcd(0x27, 16, 2);

// ADC pins
const int adcPin1 = 4; // Cell 1
const int adcPin2 = 5; // Cell 2
const int adcPin3 = 6; // Cell 3

// Bluetooth pins
#define BT_RX 15 // ESP32 RX <- Bluetooth TX
#define BT_TX 23 // ESP32 TX -> Bluetooth RX

void setup() {
  // Serial Monitor
  Serial.begin(115200);

  // Bluetooth Serial
  Serial2.begin(9600, SERIAL_8N1, BT_RX, BT_TX);

  // LCD init
  Wire.begin(21, 22);
  lcd.init();
  lcd.backlight();
  lcd.clear();
  lcd.setCursor(0, 0);
  lcd.print("ESP32 Bluetooth");
  lcd.setCursor(0, 1);
  lcd.print("Init Complete");
  delay(1500);
  lcd.clear();

  Serial.println("ESP32-C6 3 ADC Ready");
  Serial2.println("ESP32-C6 3 ADC Ready");
}

void loop() {
  // Read ADC values
  int adc1 = analogRead(adcPin1);
  int adc2 = analogRead(adcPin2);
  int adc3 = analogRead(adcPin3);

  // Convert ADC to voltage
```

```

float volt1 = (adc1 / 4095.0) * 3.3;
float volt2 = (adc2 / 4095.0) * 3.3;
float volt3 = (adc3 / 4095.0) * 3.3;

float total = volt1 + volt2 + volt3;

// Percentage (ADC based)
float perc1 = (adc1 / 4095.0) * 100.0;
float perc2 = (adc2 / 4095.0) * 100.0;
float perc3 = (adc3 / 4095.0) * 100.0;

// Prepare output string
String output =
  "C1 ADC=" + String(adc1) + " (" + String(perc1,1) + "%) V=" + String(volt1,2) +
  " | C2 ADC=" + String(adc2) + " (" + String(perc2,1) + "%) V=" + String(volt2,2) +
  " | C3 ADC=" + String(adc3) + " (" + String(perc3,1) + "%) V=" + String(volt3,2) +
  " | Total=" + String(total,2);

// Send to Serial Monitor
Serial.println(output);

// Send same data to Bluetooth
Serial2.println(output);

// LCD Display (Page 1: Cell 1 & Cell 2)
lcd.setCursor(0, 0);
lcd.print("C1:" + String((int)perc1) + "% V:" + String(volt1,1) + " ");
lcd.setCursor(0, 1);
lcd.print("C2:" + String((int)perc2) + "% V:" + String(volt2,1) + " ");
delay(2000);

// LCD Display (Page 2: Cell 3 & Total)
lcd.clear();
lcd.setCursor(0, 0);
lcd.print("C3:" + String((int)perc3) + "% V:" + String(volt3,1) + " ");
lcd.setCursor(0, 1);
lcd.print("Total: " + String(total,2) + "V ");
delay(2000);
}

```

Tasks:

1. Individual Cell Voltage Verification

Connect all three battery cells to the monitoring circuit, Read the ADC values corresponding to **ADC1, ADC2, and ADC3**, Using a multimeter, measure the actual voltage of each battery and compare it with the calculated voltage from the ADC readings, Verify that each ADC channel correctly represents its respective battery voltage.

2. ADC Scaling and Accuracy Check

Gradually vary the battery voltage (or use a variable DC supply in place of a battery), Record the ADC readings for multiple voltage levels (e.g., 3.0 V, 3.6 V, 4.2 V), Convert ADC values into voltage using the resistor divider formula, Verify that the measured voltage increases linearly with ADC count and remains within the ESP32 ADC range (0–3.3 V).

3. Voltage Divider Performance Analysis

Measure the voltage at:

- Battery positive terminal
- Divider midpoint (ADC pin)

Confirm that the divider output matches the expected theoretical value:

$$V_{ADC} = V_{battery} \times \frac{R2}{R1 + R2}$$

Discuss how resistor tolerance and ADC resolution affect measurement accuracy.

4. ADC Noise and Stability Test

Take multiple ADC readings for each battery over time.

Observe fluctuations in ADC values when:

- ESP32 is idle
- ESP32 is transmitting data (Wi-Fi / Serial active)

Discuss sources of noise and methods to reduce it (averaging, filtering, proper grounding).